

# Development of an Offline-First Point of Sale Mobile Application with Daily Sales Forecasting Using Long Short-Term Memory (LSTM)

Muhammad Kholis<sup>1</sup>, Zulfan Khairil Simbolon<sup>2\*</sup>, Azhar<sup>3</sup>, Juneidy<sup>4</sup>

<sup>1,2,3</sup> Department of Information and Computer Technology, Politeknik Negeri Lhokseumawe, Jl. Banda Aceh–Medan Km. 280.3 Buketrata, Lhokseumawe 24301, Indonesia

<sup>4</sup> Study Program of Graphic Multimedia Engineering Technology, Department of Computer and Informatics Engineering, Politeknik Negeri Medan, Jl. Almamater No. 1 USU Campus, Medan 20155, Indonesia

\*Corresponding Author: [zulfan@pnl.ac.id](mailto:zulfan@pnl.ac.id)

**Article Info:** Received August 15, 2026, Revised August 25, 2026, Accepted September 01, 2026

This is an open access article under the [CC BY-SA](https://creativecommons.org/licenses/by-sa/4.0/) license.



## Abstract

Micro, Small, and Medium Enterprises (MSMEs) frequently encounter challenges stemming from internet connectivity dependency in digital cashier systems, as well as high uncertainty in daily sales volume during inventory planning. This research designs and develops an offline-first mobile Point of Sale (POS) application integrating daily sales forecasting features powered by Long Short-Term Memory (LSTM), with an empirical case study at the EatsTEDi canteen, Vocational College, Universitas Gadjah Mada (SV UGM). The client application was engineered using Flutter and the local Isar Database, synchronized bidirectionally with cloud Supabase PostgreSQL, while the forecasting service runs via a containerized REST API (Flask/TensorFlow). Time series modeling was performed on 313 active days of real transaction records using log-return target transformation and benchmarked against an operational suitability threshold ( $sMAPE < 20\%$ ). One-step-ahead forecasting evaluation recorded an average MAE of IDR 667,718.00, RMSE of IDR 897,563.00, and  $sMAPE$  of 38.06%, indicating that the model did not reach practical operational acceptance due to data size constraints (212 training sequences) and high volatility in daily transaction patterns. Stability testing across 10 random weight initializations (multi-seed) produced an MAE spread between IDR 495,527.00 and IDR 1,048,967.00 (standard deviation of IDR 145,024.00) and an  $sMAPE$  range of 29.16%–44.63% (0 out of 10 seeds achieved the target threshold). Nevertheless, black-box functional testing across 12 core use cases, offline-to-online synchronization verification, and white-box logic path tracing across 7 critical functions achieved a 100% PASS success rate, demonstrating the robust reliability of the offline-first POS architecture in ensuring uninterrupted cashier operations for MSMEs without perpetual internet connectivity.

**Keywords:** Point of Sale, Daily Sales Forecasting, Long Short-Term Memory, Offline-First, Isar DB, MSME.

## 1. Introduction

Micro, Small, and Medium Enterprises (MSMEs) represent a vital cornerstone of national economic growth in Indonesia. Nevertheless, the majority of MSME owners still administer their daily operations through manual bookkeeping or disjointed, non-integrated recording workflows [1]. In cashier point-of-sale operations, numerous business owners encounter severe operational bottlenecks caused by continuous dependency on internet connectivity [2]. Pure cloud-based cashier applications frequently fail to function when network connectivity is disrupted, thereby halting payment processing and triggering customer queues. Therefore, an offline-first POS system architecture is urgently required, enabling cashier transactions to be executed locally with instant response times and synchronizing data automatically once network connectivity is restored [3].

Beyond cashier reliability at the checkout counter, another critical operational challenge confronting MSME managers is the unpredictability of daily sales volumes for supply chain logistics and inventory optimization [4]. The lack of

accurate demand estimates exposes businesses to substantial financial loss resulting from stockouts as well as excessive inventory accumulation leading to perishable waste (*overstocking*) [5]. In this regard, time series forecasting provides a strategic computational mechanism for quantitatively projecting operational demands [6].

Prior investigations have extensively evaluated computational intelligence models for retail sales demand forecasting across diverse operational settings. Moraes et al. engineered hybrid stacked and parallel CNN-LSTM models across 500 retail store time series spanning five years, yielding MAPE values of 14.15% and 14.76%, respectively [7]. Mansur et al. proposed a CNN-LSTM architecture incorporating seven exogenous variables (such as weather and public holidays) across six months of retail sales data, achieving a MAPE of 4.16% and outperforming standalone ANN (19.74%) and single LSTM (10.39%) models [8]. Mitra et al. implemented LSTM neural networks with Keras Tuner hyperparameter optimization on commercial retail datasets, evidencing notable error reduction compared to traditional baselines [9]. Yadav and Barde developed a real-time web-based sales prediction pipeline combining LSTM and XGBoost linked to POS registers [10]. In Indonesian retail studies, Sembiring and Sary deployed LSTM for warehouse inventory replenishment, reporting MAPE values of 102–106% compared to 85–88% for Moving Average baselines [11], while Jannah et al. compared LSTM and SARIMA on commercial water refill businesses [12]. Furthermore, Verdianto et al. engineered an MSME beverage POS application embedded with Random Forest Regression [13].

Although deep learning architectures such as LSTM are conceptually designed to capture complex non-linear temporal dependencies across sequential data [14], [15], findings by Makridakis et al. in the M5 Competition underscored that simple statistical benchmarks often remain highly competitive against intricate neural networks when applied to short or volatile time series [16]. In micro-enterprise environments characterized by restricted historical records, time series models remain exceptionally sensitive to academic calendar breaks and weight initialization instability. Furthermore, previous studies have rarely benchmarked model robustness against rigorous operational feasibility thresholds or integrated forecasting inference directly into an offline-first mobile POS platform.

To address these empirical and architectural gaps, this study designs and implements an offline-first mobile Point of Sale system built with Flutter and Isar DB connected to Supabase PostgreSQL, integrated with a daily sales forecasting microservice powered by an LSTM neural network deployed via Flask on Hugging Face Spaces. The study conducts an in-depth empirical performance evaluation using both one-step-ahead forecasting and multi-seed stability testing against a practical operational threshold ( $sMAPE < 20\%$ ) using 313 active days of real transaction records from the EatsTED! SV UGM canteen. System verification was rigorously performed using black-box functional testing, offline synchronization validation, and white-box control path analysis across mission-critical software routines.

## 2. Research Methods

### 2.1 Research Workflow

This research was conducted under a quantitative Research and Development (R&D) paradigm. The methodology comprises two interconnected core phases: the design and engineering of the offline-first Point of Sale application, and the development and empirical evaluation of the LSTM time series forecasting model, as depicted in Figure 1.

### 2.2 Offline-First POS System Architecture

The system architecture was designed to guarantee uninterrupted cashier transactions without requiring persistent network connectivity. The platform consists of four primary components (Figure 1):

- Mobile Client Application: Built with Flutter (Dart programming language) following the Riverpod reactive state management architecture [3], [20]. Local persistence is managed by Isar Database [21], which stores product catalogs, categories, and receipts instantaneously. A background Sync Engine periodically monitors network connectivity; upon detecting an active internet connection, it dispatches unsynchronized local transaction records marked with `is_synced = false` to the cloud database.
- Cloud Infrastructure (Supabase): Utilizes managed Supabase PostgreSQL as a centralized relational database equipped with Row Level Security (RLS) policies [22] and a stored procedure (*Remote Procedure Call*/RPC) named `create_transaction_v4` to execute atomic transaction records and inventory deductions.
- Telemetry and Messaging Services (Firebase): Employs Firebase Cloud Messaging (FCM) for broadcast announcements and low-stock push alerts, and Firebase Analytics for in-app operational telemetry.
- Predictive Inference Service (Hugging Face): The trained LSTM model is served via a Flask REST API on Hugging Face Spaces. The mobile client transmits aggregated historical revenue sequences via HTTPS and receives multi-

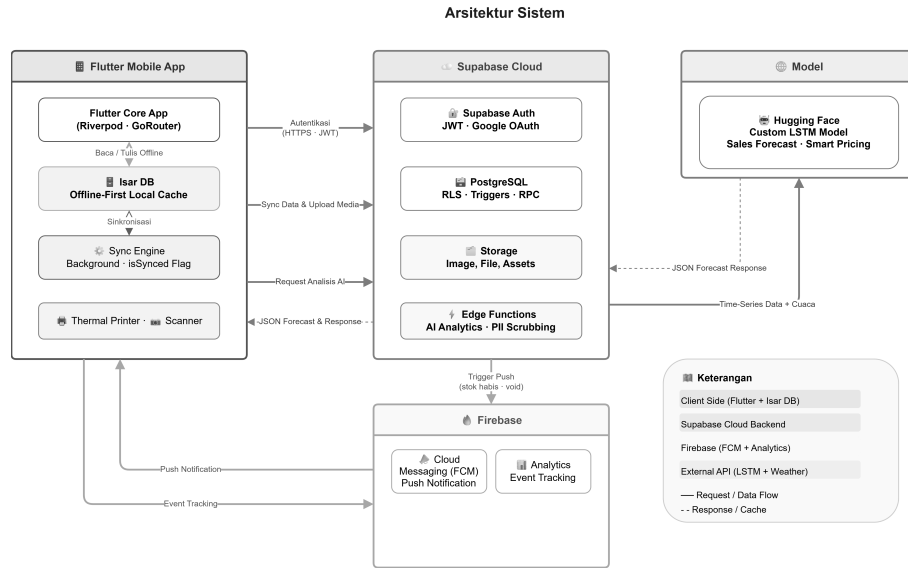


Figure 1. Architecture of the Offline-First Mobile POS System Integrated with LSTM Forecasting Services

step forecasting projections in structured JSON format.

### 2.3 Dataset and Data Preprocessing

The empirical dataset comprises authentic transactional records collected from the EatsTEDi SV UGM canteen between October 26, 2024, and September 4, 2025. Preprocessing was conducted through the following stages:

- Active Day Filtering: Retained only operational business days with positive gross revenue (revenue > 0), resulting in 313 active sales days. Academic holiday closures and non-trading weekends were pruned to establish a continuous operational sequence.
- Target Log-Return Transformation: To mitigate non-stationarity and stabilize wide revenue variance, the target series was converted into daily log-returns ( $r_t$ ):

$$r_t = \ln \left( \frac{Y_t + 1}{Y_{t-1} + 1} \right) \quad (1)$$

where  $Y_t$  represents the nominal gross sales revenue on active day  $t$ . The  $+1$  offset prevents undefined values during zero-revenue anomalies.

- Cyclical Temporal Features: Extracted day-of-week calendar cycles using sine and cosine trigonometric transformations:

$$\sin_{\text{dow}} = \sin \left( \frac{2\pi \cdot d}{7} \right), \quad \cos_{\text{dow}} = \cos \left( \frac{2\pi \cdot d}{7} \right) \quad (2)$$

where  $d \in \{0, 1, \dots, 6\}$  designates the day index within the weekly cycle.

- Normalization and Sequence Construction: Continuous numerical features were scaled into  $[0, 1]$  via Min-Max Scaling [23]. Supervised learning samples were generated using a sliding window across 7 active trading days ( $\text{look\_back} = 7$ ), resulting in an input tensor dimension of  $7 \times 8$ .
- Chronological Dataset Splitting: Data was partitioned chronologically (time-based split) into 70% training set (212 sequences), 15% validation set (45 sequences), and 15% testing set (46 sequences).

## 2.4 Long Short-Term Memory (LSTM) Model Architecture

The LSTM recurrent neural network was employed to learn temporal sequential patterns from sales history [14]. Within each LSTM cell at time step  $t$ , state transitions are governed by four gating mechanisms:

$$\mathbf{f}_t = \sigma(\mathbf{W}_f \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f) \quad (3)$$

$$\mathbf{i}_t = \sigma(\mathbf{W}_i \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i) \quad (4)$$

$$\tilde{\mathbf{C}}_t = \tanh(\mathbf{W}_C \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_C) \quad (5)$$

$$\mathbf{C}_t = \mathbf{f}_t \odot \mathbf{C}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{C}}_t \quad (6)$$

$$\mathbf{o}_t = \sigma(\mathbf{W}_o \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o) \quad (7)$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{C}_t) \quad (8)$$

where  $\mathbf{f}_t$ ,  $\mathbf{i}_t$ ,  $\mathbf{o}_t$  denote the forget gate, input gate, and output gate vectors, respectively;  $\mathbf{C}_t$  represents the internal cell state;  $\mathbf{h}_t$  denotes the hidden state;  $\sigma(\cdot)$  is the logistic sigmoid activation; and  $\odot$  represents element-wise multiplication (Hadamard product).

The layer configuration of the Hybrid LSTM model is summarized in Algorithm 1, and the training hyperparameters are listed in Table 1. Optimization was carried out using the Adam algorithm [24], complemented by dropout regularization [25] to prevent model overfitting.

---

### Algorithm 1 Construction of Hybrid Residual LSTM Model Using TensorFlow/Keras

---

**Require:** Input sequences  $\mathbf{X}_t \in \mathbb{R}^{7 \times 8}$  ( $look\_back = 7$ ,  $n\_features = 8$ ).

**Ensure:** Predicted log-return of daily sales revenue  $\hat{r}_t \in \mathbb{R}$ .

```

1: model ← Sequential()
2: model.add(LSTM(units = 24, input_shape = (7, 8), return_sequences = False))
3: model.add(Dropout(rate = 0.1))
4: model.add(Dense(units = 16, activation = 'relu'))
5: model.add(Dense(units = 1, activation = 'linear'))
6: model.compile(optimizer = Adam(lr = 5 × 10-4), loss = 'mse', metrics = ['mae'])
7: return model

```

---

Table 1. Training Hyperparameter Specifications for the LSTM Model

| Hyperparameter             | Value / Specification                                        |
|----------------------------|--------------------------------------------------------------|
| Input Tensor Shape         | $7 \times 8$ ( $LOOK\_BACK = 7$ active days, 8 features)     |
| LSTM Hidden Layer          | 1 layer, 24 LSTM cell units                                  |
| Dropout Rate               | 0.1                                                          |
| Fully Connected Layer      | Dense 16 units (ReLU activation)                             |
| Output Layer               | Dense 1 linear unit (log-return prediction)                  |
| Total Trainable Parameters | 3,585 parameters                                             |
| Optimization Algorithm     | Adam (learning rate = $5 \times 10^{-4}$ )                   |
| Loss Function              | Mean Squared Error (MSE)                                     |
| Batch Size                 | 16                                                           |
| Maximum Epochs             | 200 (Early Stopping <i>patience</i> =20 & ReduceLROnPlateau) |

---

## 2.5 Nominal Reconstruction and Multi-Step Projection

The predicted log-return  $\hat{r}_t$  was mapped back into nominal currency scale (IDR,  $\hat{Y}_t$ ) via inverse exponential reconstruction:

$$\hat{Y}_t = \exp(\ln(Y_{t-1} + 1) + \hat{r}_t) - 1 \quad (9)$$

For recursive multi-step forecasting extending up to 7 active business days ahead, three safety guards were enforced:

- Clipping: Bound forecasted log-returns within the 10th and 90th empirical percentiles of the training distribution to prevent extreme deviations.
- Damping: Scaled projected variations by an exponential decay factor  $\phi^{h-1}$  ( $\phi = 0.7$ ) as the step horizon  $h$  increases.
- Safety Rail: Clamped projected nominal sales revenue to never exceed the historical upper bound (revenue ceiling).

## 2.6 Evaluation Metrics and Feasibility Benchmark

Forecasting accuracy was examined under a one-step-ahead protocol utilizing three complementary evaluation metrics [6], [26]:

$$MAE = \frac{1}{N} \sum_{t=1}^N |Y_t - \hat{Y}_t| \quad (10)$$

$$RMSE = \sqrt{\frac{1}{N} \sum_{t=1}^N (Y_t - \hat{Y}_t)^2} \quad (11)$$

$$sMAPE = \frac{100\%}{N} \sum_{t=1}^N \frac{|Y_t - \hat{Y}_t|}{(|Y_t| + |\hat{Y}_t|)/2} \quad (12)$$

where  $Y_t$  denotes actual gross revenue,  $\hat{Y}_t$  is the forecasted revenue, and  $N$  represents the test sample size ( $N = 46$ ). Symmetric MAPE (sMAPE) serves as the primary benchmark because it provides bounded, scale-free percentage errors without diverging when encountering small sales values. The practical operational threshold was set at  $sMAPE < 20\%$ .

## 3. Results and Discussion

### 3.1 Evaluation of LSTM Forecasting Model Against Practical Threshold

Model performance on the test dataset (46 active days) was assessed across two rigorous evaluation scenarios. The first scenario benchmarked the one-step-ahead forecasting accuracy of the Hybrid LSTM model. The empirical evaluation metrics are summarized in Table 2.

Table 2. Hybrid LSTM Model Performance on Test Dataset (One-Step-Ahead)

| Model / Architecture                   | MAE (IDR) | RMSE (IDR) | sMAPE (%)          |
|----------------------------------------|-----------|------------|--------------------|
| Hybrid LSTM (10-Seed Average)          | 667,718   | 897,563    | 38.06%             |
| <b>Practical Feasibility Threshold</b> | —         | —          | <b>&lt; 20.00%</b> |

As detailed in Table 2, the Hybrid LSTM model yielded an average MAE of IDR 667,718.00, RMSE of IDR 897,563.00, and sMAPE of 38.06%. The sMAPE metric substantially exceeded the pre-established 20% operational feasibility benchmark. The visual comparison between actual revenue and model forecasts is illustrated in Figure 2.

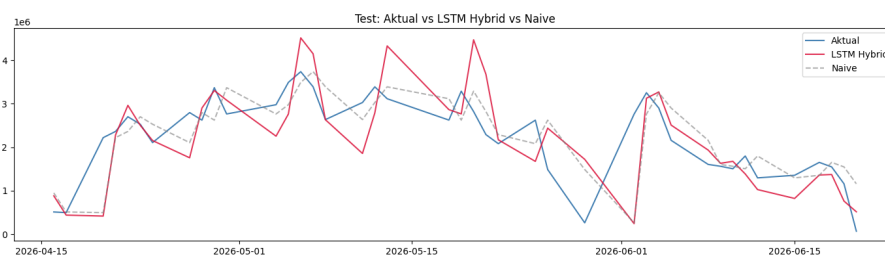


Figure 2. Time Series Comparison of Actual Canteen Revenue Against Hybrid LSTM Model Forecasts

The trajectory in Figure 2 highlights that the LSTM model exhibited pronounced over-smoothing. While the model successfully approximated medium-term central revenue tendencies, it was unable to capture sudden, high-magnitude sales peaks induced by campus events and irregular academic schedules.

### 3.2 Model Stability Testing via Multi-Seed Evaluation

The second evaluation scenario tested model stability against random weight initialization variations. The training pipeline was executed 10 times under identical parameters using distinct random seeds ( $seed \in \{0, 1, \dots, 9\}$ ). Descriptive performance statistics across the 10 runs are provided in Table 3.

The empirical results in Table 3 reveal considerable variance across seeds, exhibiting a standard deviation of IDR 145,024.00 in MAE and 3.96% in sMAPE. The worst-performing initialization (MAE IDR 1,048,967.00) recorded an error more than double that of the best initialization (MAE IDR 495,527.00). Crucially, none of the 10 seeds reached

Table 3. Hybrid LSTM Performance Across Multi-Seed Stability Testing (10 Runs)

| Evaluation Metric | Mean    | Std. Deviation ( $\pm$ ) | Range (Min – Max)   |
|-------------------|---------|--------------------------|---------------------|
| MAE (IDR)         | 667,718 | 145,024                  | 495,527 – 1,048,967 |
| RMSE (IDR)        | 897,563 | 165,804                  | 703,721 – 1,320,587 |
| sMAPE (%)         | 38.06%  | 3.96%                    | 29.16% – 44.63%     |

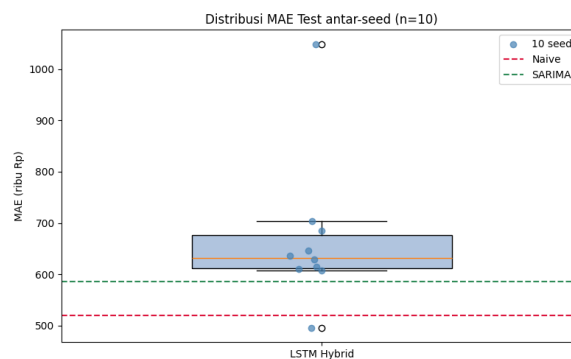


Figure 3. MAE Error Distribution of Hybrid LSTM Across Multi-Seed Stability Runs (10 Iterations)

the target sMAPE threshold of  $< 20\%$  (the top-performing run achieved 29.16% on Seed 7). The distribution of MAE values across runs is depicted as a boxplot in Figure 3.

This dispersion underscores that training neural networks on modest sequential datasets (212 training samples) is inherently prone to initialization instability (*seed variance*). Consequently, reporting single-run performance for deep forecasting architectures on small datasets introduces significant reporting bias. A concise synthesis of both evaluation scenarios is presented in Table 4.

Table 4. Summary of Empirical Evaluation Scenarios for the Forecasting Model

| Scenario | Evaluation Objective                                                | Threshold Met? | Key Empirical Finding                                                                                                 |
|----------|---------------------------------------------------------------------|----------------|-----------------------------------------------------------------------------------------------------------------------|
| 1        | Measure one-step-ahead accuracy against benchmark (sMAPE $< 20\%$ ) | No             | Mean MAE IDR 667,718.00 and sMAPE 38.06%, exceeding operational threshold.                                            |
| 2        | Assess training stability across 10 randomized seeds                | No             | Wide MAE spread (IDR 495,527–1,048,967) with standard deviation IDR 145,024; 0 of 10 seeds satisfied sMAPE $< 20\%$ . |

### 3.3 Mobile Point of Sale Interface Implementation

The mobile POS application was deployed to Android mobile hardware featuring a modern, intuitive user interface. The system delivers complete cashier transaction workflows, catalog management, inventory adjustments, daily revenue dashboards, and a specialized Smart Analytics screen presenting 7-day predictive projections generated by the LSTM inference microservice (Figure 4).

The Smart Analytics module displays projected gross sales, multi-horizon trend charts with ceiling limits (safety rails), and cold-start detection banners that advise store operators when sales history has not yet reached the required 30-day warm-up baseline.

### 3.4 Functional System Testing (Black Box Testing)

Comprehensive black-box verification was executed across 12 core operational use cases covering both Owner and Cashier roles in compliance with software testing standards [17], [18], [19]. Every test case completed successfully with a 100% PASS outcome (Table 5).

### 3.5 Evaluation of Offline-First Synchronization Mechanics

In-depth testing of the offline-first architecture was conducted to ensure data integrity during offline-to-online transitions. Results are presented in Table 6.

These findings demonstrate that embedding Isar DB provides near-instant cashier responsiveness while preserving total transactional integrity during network outages.

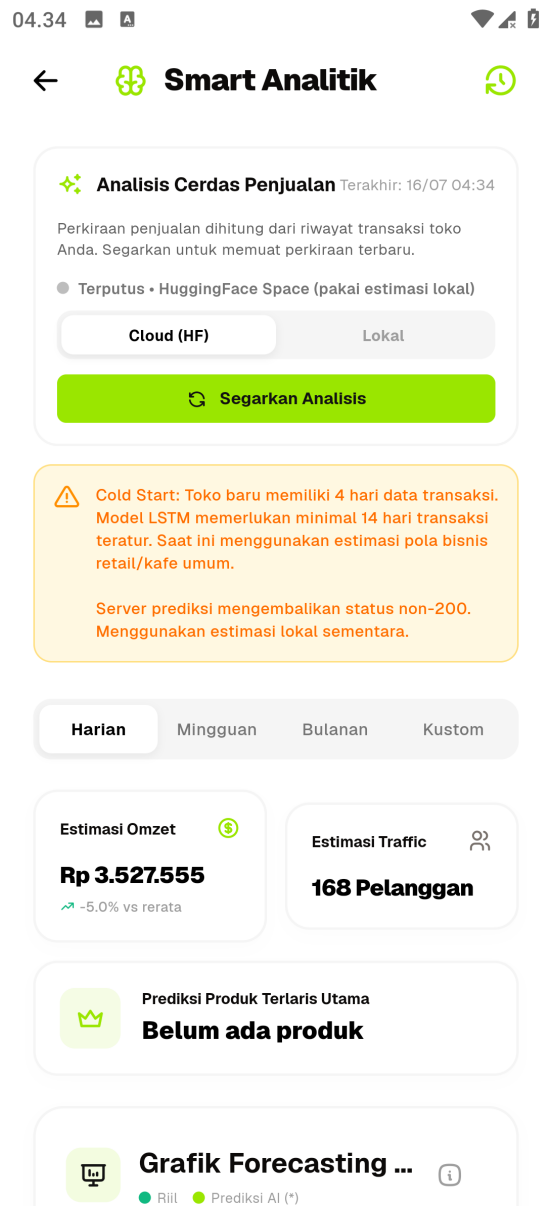


Figure 4. User Interface of the Smart Analytics Module in the Mobile POS Application

Table 5. Black Box Test Outcomes Across Core Mobile POS Features

| No | Feature / Test Case   | Testing Action                             | Expected Result                            | Status |
|----|-----------------------|--------------------------------------------|--------------------------------------------|--------|
| 1  | Merchant Registration | Enter merchant profile & store credentials | Account registered on cloud PostgreSQL     | PASS   |
| 2  | User Authentication   | Submit valid credentials                   | Dashboard access matched to user role      | PASS   |
| 3  | Checkout Transaction  | Select items & confirm payment             | Local commit & immediate stock deduction   | PASS   |
| 4  | Product Management    | Create, update, and delete menu items      | Catalog updated instantly in local DB      | PASS   |
| 5  | Category Organization | Organize menu category taxonomies          | Categories saved and displayed at register | PASS   |
| 6  | Inventory Stock Audit | Review stock adjustment history log        | Accurate records of additions/deductions   | PASS   |
| 7  | Sales Dashboard       | Review daily turnover summaries            | Valid aggregation & sales charts           | PASS   |
| 8  | Smart Analytics       | Request 7-day sales projections            | Projections displayed with bounds          | PASS   |
| 9  | Staff Notifications   | Dispatch broadcast message to staff        | FCM notification received on cashier app   | PASS   |
| 10 | Data Synchronization  | Restore internet connectivity              | Queued transactions synced to cloud        | PASS   |
| 11 | Staff Management      | Configure cashier accounts & permissions   | Cashier role verified and updated          | PASS   |
| 12 | Store Profile         | Update business name, address, and logo    | Profile propagated across all modules      | PASS   |

Table 6. Validation Results of Offline-First Synchronization (Isar DB to Supabase)

| No | Test Scenario                 | Observed System Behavior                                                                                     | Status |
|----|-------------------------------|--------------------------------------------------------------------------------------------------------------|--------|
| 1  | Checkout during offline state | Receipt saved in local Isar DB with <code>is_synced = false</code> ; inventory reduced locally.              | PASS   |
| 2  | View sales history offline    | Instant retrieval of local sales without network latency or UI blocking.                                     | PASS   |
| 3  | Reconnection to internet      | Sync Engine detects online status and uploads queued receipts via Supabase RPC.                              | PASS   |
| 4  | Verify cloud data consistency | Transactions committed to Supabase without duplicates; local flag updated to <code>is_synced = true</code> . | PASS   |

### 3.6 Code Logic Path Testing (White Box Testing)

White-box unit testing was conducted to verify critical conditional branches in the Python forecasting API service (Table 7).

Table 7. White Box Test Verification of Critical Forecasting API Functions

| No | Function / Target Path    | Input Test Condition                 | Expected Execution Outcome                             | Status |
|----|---------------------------|--------------------------------------|--------------------------------------------------------|--------|
| 1  | by_dow (Seasonal-Naive)   | Historical day-of-week data $\geq k$ | Computes mean across recent $k$ matching days          | PASS   |
| 2  | by_dow (Partial Branch)   | Historical day-of-week data $< k$    | Computes mean across all available matching days       | PASS   |
| 3  | by_dow (Fallback Path)    | Day-of-week completely missing       | Falls back to mean of last 10 active days              | PASS   |
| 4  | smape (Zero Denominator)  | Actual and forecast values are zero  | Denominator defaulted to 1; zero-division avoided      | PASS   |
| 5  | forecast (Clipping Guard) | Log-return exceeds percentile bound  | Value clipped at empirical upper percentile            | PASS   |
| 6  | forecast (Damping Decay)  | Forecast step $h > 1$                | Variance decayed by multiplicative factor $\phi^{h-1}$ | PASS   |
| 7  | forecast (Safety Rail)    | Forecast exceeds historical max      | Nominal forecast clamped to historical ceiling         | PASS   |

The white-box verification confirmed that all edge case exception routines and safety clamping logic functioned without runtime errors.

### 3.7 Discussion and Findings Positioning

Empirical findings demonstrate that the forecasting shortfall ( $sMAPE = 38.06\%$ ) stems directly from the limited volume of historical training records (313 active days, 212 training sequences) paired with the sharp demand volatility characteristic of campus cafeteria environments. This aligns with findings from Makridakis et al. during the M5 Competition, where deep neural networks provided minimal competitive advantage over simpler statistical heuristics on brief time series lacking multi-year cyclical repetitions [16]. Furthermore, eliminating non-trading academic holidays (revenue = 0) created irregular calendar gaps between active business days, challenging the continuous time-step assumptions of recurrent neural architectures [5].

Nonetheless, this research contributes key engineering and methodological insights to applied predictive software development:

- Multi-Seed Evaluation Transparency: Discloses neural network initialization variance through rigorous reporting of standard deviations and error bounds.
- Validity of  $sMAPE$  Metric: Prevents exploding percentage error artifacts encountered by standard MAPE when analyzing small-enterprise sales with near-zero baselines.
- Resilient Offline-First Architecture: Demonstrates the practical feasibility of pairing local Isar DB persistence with Supabase PostgreSQL to achieve zero cashier downtime regardless of connectivity status.

## 4. Conclusion and Future Work

### 4.1 Conclusion

This research successfully engineered an offline-first mobile Point of Sale application utilizing Flutter, Isar Database, Supabase PostgreSQL, and a containerized LSTM forecasting REST microservice hosted on Hugging Face Spaces. Black-box functional validation across 12 primary use cases, offline synchronization verification, and white-box unit tests all achieved a 100% PASS rate, confirming the system's ability to ensure zero downtime and near-instant checkout operations for MSMEs without continuous network connectivity. Regarding predictive modeling, empirical

evaluation of the Hybrid LSTM model on 313 active days yielded an average MAE of IDR 667,718.00, RMSE of IDR 897,563.00, and sMAPE of 38.06%, exceeding the practical feasibility threshold (sMAPE < 20%). Multi-seed stability testing demonstrated substantial variance across runs (MAE IDR 495,527.00–1,048,967.00; sMAPE 29.16%–44.63%), indicating that deep recurrent architectures remain highly sensitive to weight initialization and require larger datasets to achieve stable convergence in micro-retail settings.

## 4.2 Future Work

Future research should expand data collection horizons across three to five operational years to enable deep sequence models to capture multi-year seasonal cycles effectively. Incorporating elapsed calendar gap features between non-consecutive trading days could alleviate temporal distortions caused by academic breaks. Additionally, future initiatives may explore transfer learning or multi-tenant global architectures across comparable retail categories to mitigate cold-start limitations, as well as investigate hybrid combinations (e.g., SARIMA-LSTM) that unify linear seasonal modeling with non-linear adaptation. Finally, enabling cryptographic encryption on the local Isar Database is recommended to further strengthen device-level data protection.

## References

- [1] E. M. Sipayung, C. Fiarni, and Wawan, "Evaluation of Point of Sale Application Usage Using Technology Acceptance Model in MSMEs," *Jurnal Nasional Teknik Elektro dan Teknologi Informasi (JNTETI)*, vol. 9, no. 1, pp. 18–24, 2020, doi: 10.22146/jnteti.v9i1.1116.
- [2] M. Siddik and S. Samsir, "Design and Implementation of POS (Point of Sale) Information System for Cashiers Using Object-Oriented Programming Concepts," *JOISIE (Journal of Information Systems and Informatics Engineering)*, vol. 4, no. 1, pp. 43–49, 2020, doi: 10.35145/joisie.v4i1.607.
- [3] Google LLC, "Flutter Documentation: Offline-first Architecture and Local Persistence," 2026. [Online]. Available: <https://docs.flutter.dev/>. [Accessed: 29-Jul-2026].
- [4] R. J. Hyndman and G. Athanasopoulos, *Forecasting: Principles and Practice*, 3rd ed. Melbourne, Australia: OTexts, 2021.
- [5] G. E. P. Box, G. M. Jenkins, G. C. Reinsel, and G. M. Ljung, *Time Series Analysis: Forecasting and Control*, 5th ed. Hoboken, NJ, USA: John Wiley & Sons, 2015, doi: 10.1002/9781118619193.
- [6] R. J. Hyndman and A. B. Koehler, "Another Look at Measures of Forecast Accuracy," *International Journal of Forecasting*, vol. 22, no. 4, pp. 679–688, 2006, doi: 10.1016/j.ijforecast.2006.03.001.
- [7] T. de Castro Moraes, X.-M. Yuan, and E. P. Chew, "Hybrid Convolutional Long Short-Term Memory Models for Sales Forecasting in Retail," *Journal of Forecasting*, vol. 43, no. 5, pp. 1278–1293, 2024, doi: 10.1002/for.3073.
- [8] S. Mansur, K. Sattar, S. E. Hosseini, S. Pervez, I. Ahmad, K. Saleem, and A. Z. Elhendi, "Sales Forecasting for Retail Stores Using Hybrid Neural Networks and Sales-Affecting Variables," *PeerJ Computer Science*, vol. 11, Art. no. e3058, 2025, doi: 10.7717/peerj-cs.3058.
- [9] M. Mitra, S. Roy, S. De, S. Bhattacharyya, J. Platos, and V. Snasel, "Harnessing LSTM Neural Networks and Hyperparameter Optimization for Precise Sales Forecasting in Retail," in *Proc. 10th Int. Conf. Adv. Intell. Syst. Inform. (AISI 2024)*, ser. Lecture Notes on Data Engineering and Communications Technologies, vol. 220, Cham, Switzerland: Springer, 2024, pp. 120–131, doi: 10.1007/978-3-031-71619-5\_11.
- [10] P. Yadav and C. R. Barde, "Retail Real-Time Sales Prediction System Using LSTM and XGBoost," *International Journal of Advanced Research in Computer and Communication Engineering (IJARCCE)*, vol. 14, no. 4, pp. 1–9, 2025, doi: 10.17148/IJARCCE.2025.14401.
- [11] R. A. Sembiring and Y. Sary, "Analysis and Implementation of Long Short-Term Memory (LSTM) for Inventory Demand in PT. Gunung Sari Indonesia," *Jurnal Ilmiah Teknik Mesin, Elektro dan Komputer*, vol. 5, no. 1, pp. 112–124, 2025.
- [12] U. M. Jannah, D. S. Hormansyah, and D. P. Hapsari, "Analysis of Water Refill Sales Prediction Using LSTM and SARIMA Methods," *Jurnal Teknologi Informasi dan Multimedia*, vol. 7, no. 2, pp. 88–97, 2025.
- [13] R. Verdianto, D. Hartanti, and E. Purwanto, "Development of Point of Sales Application for Beverage Enterprise Daily Sales Prediction Using Random Forest Regression Algorithm," *Infotek: Jurnal Informatika dan Teknologi*, vol. 8, no. 2, pp. 310–321, 2025, doi: 10.29408/jit.v8i2.25681.
- [14] S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997, doi: 10.1162/neco.1997.9.8.1735.
- [15] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA, USA: MIT Press, 2016.
- [16] S. Makridakis, E. Spiliotis, and V. Assimakopoulos, "The M5 Competition: Background, Organization, and Implementation," *International Journal of Forecasting*, vol. 38, no. 4, pp. 1325–1336, 2022, doi: 10.1016/j.ijforecast.2021.07.007.
- [17] R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner's Approach*, 9th ed. New York, NY, USA: McGraw-Hill Education, 2020.
- [18] G. J. Myers, C. Sandler, and T. Badgett, *The Art of Software Testing*, 3rd ed. Hoboken, NJ, USA: John Wiley & Sons, 2011, doi: 10.1002/9781119202486.
- [19] *Software and Systems Engineering — Software Testing — Part 4: Test Techniques*, ISO/IEC/IEEE Standard 29119-4:2021, 2021.
- [20] R. Rousselet, "Riverpod: A Reactive Caching and Data-Binding Framework for Flutter," 2023. [Online]. Available: <https://riverpod.dev/>. [Accessed: 29-Jul-2026].
- [21] S. Leichsenring, "Isar Database: Extremely Fast, Easy to Use, and Flexible NoSQL Database for Flutter," 2024. [Online]. Available: <https://isar.dev/>. [Accessed: 29-Jul-2026].
- [22] Supabase Inc., "Supabase Documentation: PostgreSQL Row Level Security and Stored Procedures," 2024. [Online]. Available: <https://supabase.com/docs>. [Accessed: 29-Jul-2026].

- [23] F. Pedregosa *et al.*, "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [24] D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," in *Proc. 3rd Int. Conf. Learn. Representations (ICLR 2015)*, San Diego, CA, USA, 2015, pp. 1–15.
- [25] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A Simple Way to Prevent Neural Networks from Overfitting," *Journal of Machine Learning Research*, vol. 15, no. 56, pp. 1929–1958, 2014.
- [26] T. O. Hodson, "Root-Mean-Square Error (RMSE) or Mean Absolute Error (MAE): When to Use Them or Not," *Geoscientific Model Development*, vol. 15, no. 20, pp. 5481–5487, 2022, doi: 10.5194/gmd-15-5481-2022.